LAPORAN PRAKTIKUM BASIS DATA

MODUL 10: DBMS DAN APLIKASI BASIS DATA

1. IDENTITAS MAHASISWA & KELOMPOK

Atribut	Keterangan
Nama Mahasiswa	KHANSA QAIS FERNANDA
NIM	264311019
Kelas	TRPL 1A
Kelompok	KELOMPOK MAC OS
Studi Kasus	Implementasi Basis Data Penjualan Sederhana (praktikum_modul10)
Sub-CPMK	SCPMK0910809 – Mampu mengimplementasikan basis data menggunakan DBMS

Penilaian Keaktifan dan Partisipasi Individu Anggota Kelompok (Skala 1 – 4)

No	Nama Anggota	NIM	Kelas	Peran pada Praktikum Modul 10	Skor (1-4)
1	Khansa Qais Fernanda	264311019	TRPL 1A	Instalasi & konfigurasi MySQL 8.4.3, pembuatan skema DDL/DML, simulasi 3,1 juta baris, analisis EXPLAIN, backup/restore, Git	4 (Sangat Baik)
2	Jelita Palupi	264311017	TRPL 1A	Eksplorasi arsitektur DBMS, pengujian operasi CRUD, dan pengujian constraint/integritas referensial	4 (Sangat Baik)
3	Zahro Choirun Latifah	264311030	TRPL 1A	Pengujian transaksi ACID (COMMIT & ROLLBACK), dokumentasi hasil praktikum, dan penyusunan lembar kerja	4 (Sangat Baik)

10.2 JAWABAN SOAL PRE-TEST

1. Jelaskan apa yang dimaksud dengan DBMS dan apa perbedaannya dengan aplikasi pengelola basis data seperti phpMyAdmin atau MySQL Workbench! (Bobot: 30)

- **Pengertian DBMS:** Database Management System (DBMS) adalah perangkat lunak inti (database server) yang bertugas menyimpan data ke media penyimpanan, memproses query SQL, mengatur transaksi ACID, menjaga integritas referensial, mengatur konkurensi, serta melakukan pemulihan data (recovery). Contohnya: MySQL Server (mysqld.exe), MariaDB, PostgreSQL.

- **Perbedaan dengan Aplikasi Pengelola:** DBMS adalah mesin server yang bekerja di belakang layar (misal berjalan pada port TCP 3306), sedangkan aplikasi pengelola basis data (Database Management Client) seperti phpMyAdmin, MySQL Workbench, DBeaver, dan HeidiSQL adalah antarmuka (GUI/Web) yang digunakan pengguna untuk terhubung ke DBMS dan mengirimkan perintah SQL. Tanpa DBMS yang berjalan, aplikasi pengelola tidak dapat menyimpan atau memproses data.

2. Apa fungsi Primary Key dan Foreign Key dalam menjaga integritas data pada basis data relasional? (Bobot: 30)

- **Fungsi Primary Key (PK):** Menjaga Entity Integrity dengan memastikan setiap baris di dalam tabel memiliki identitas yang unik dan tidak boleh bernilai kosong (NOT NULL), sehingga tidak pernah terjadi duplikasi baris data induk.
- **Fungsi Foreign Key (FK):** Menjaga Referential Integrity dengan memastikan nilai kunci tamu pada tabel anak selalu merujuk pada nilai Primary Key yang benar-benar ada pada tabel induk. FK mencegah masuknya transaksi tanpa induk yang sah (orphan record) dan mencegah penghapusan data induk yang masih memiliki riwayat transaksi terkait (melalui aturan ON DELETE RESTRICT).

3. Mengapa penggunaan indeks dapat membantu meningkatkan performa query, dan apakah semakin banyak indeks selalu membuat basis data semakin cepat? Jelaskan! (Bobot: 40)

- **Alasan Indeks Meningkatkan Performa Query:** Indeks membangun struktur data terurut (umumnya B-Tree pada InnoDB) yang menyimpan nilai kolom beserta pointer ke lokasi baris aslinya. Saat mencari data dengan klausa WHERE, JOIN, atau ORDER BY, query optimizer dapat langsung menelusuri pohon indeks secara logaritmik O(log N) tanpa perlu membaca seluruh baris tabel satu per satu (Full Table Scan).
- **Apakah Semakin Banyak Indeks Selalu Lebih Cepat?** Tidak. Terlalu banyak indeks justru menurunkan performa operasi tulis (INSERT, UPDATE, DELETE) karena setiap kali ada perubahan data, DBMS wajib memperbarui seluruh struktur pohon B-Tree indeks yang ada di tabel tersebut, serta memakan ruang penyimpanan (disk) dan memori (buffer pool) tambahan.

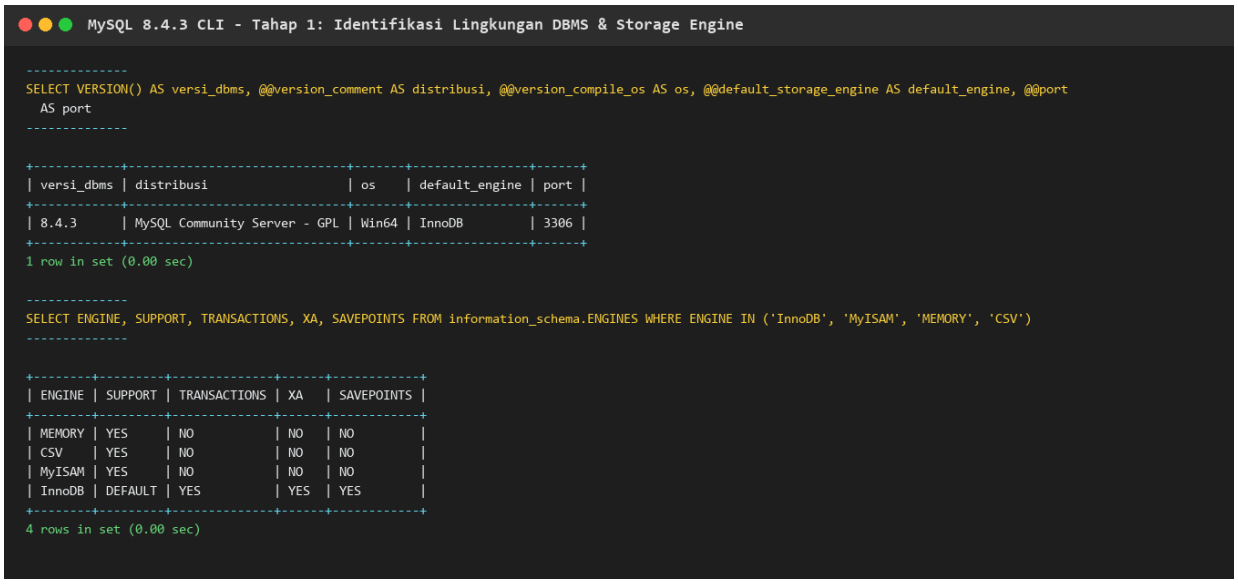
2. TAHAP 1 – IDENTIFIKASI DBMS DAN ARSITEKTUR SISTEM

2.1 Hasil Identifikasi Lingkungan DBMS

Perintah SQL yang dijalankan untuk memeriksa versi DBMS dan daftar storage engine:

```
SELECT VERSION() AS versi_dbms, @@version_comment AS distribusi, @@version_compile_os AS os,
@@default_storage_engine AS default_engine, @@port AS port;
SELECT ENGINE, SUPPORT, TRANSACTIONS, XA, SAVEPOINTS FROM information_schema.ENGINES WHERE
ENGINE IN ('InnoDB', 'MyISAM', 'MEMORY', 'CSV');
```

Parameter Lingkungan	Hasil Identifikasi pada Mesin Praktikum
DBMS / Database Server	MySQL Community Server - GPL (mysqld.exe)
Versi DBMS	8.4.3 (Win64 pada Windows)
Port & Protokol	TCP Port 3306 (MySQL Classic Protocol) & Port 33060 (X Plugin)
Default Storage Engine	InnoDB (Mendukung Transactions=YES, XA=YES, Savepoints=YES, Foreign Key)
Lokasi Data (datadir)	C:/laragon/data/mysql-8.4
Aplikasi Pengelola (Client)	phpMyAdmin (Web), HeidiSQL, DBeaver 26.2.0, serta MySQL CLI (mysql.exe & mysqldump.exe)



Gambar 1. Bukti Identifikasi Versi MySQL 8.4.3 dan Storage Engine InnoDB

2.2 Matriks Pemilihan Perangkat Lunak Pengelola Basis Data

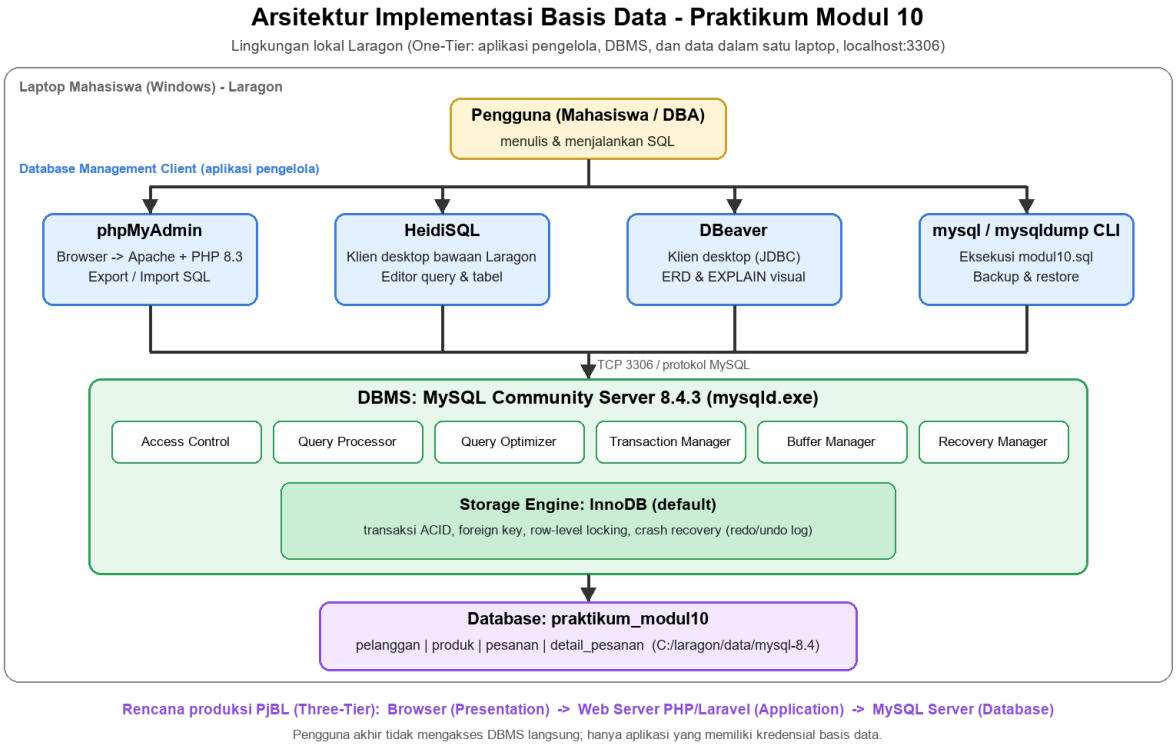
Kriteria	phpMyAdmin	HeidiSQL	DBeaver	MySQL CLI (mysql/mysqldump)
Kompatibilitas	Berjalan di semua OS via web browser (Apache + PHP)	Native Windows (ringan, bawaan Laragon)	Multi-platform (Windows, macOS, Linux) berbasis Java	Bawaan resmi instalasi MySQL di semua OS
Kemudahan Penggunaan	Sangat mudah untuk pemula, antarmuka web intuitif	Ringan, cepat dibuka, tabel & data mudah diedit	Antarmuka lengkap untuk proyek besar & ERD	Berbasis teks (perlu hafal perintah CLI)
Pengelolaan SQL	Editor SQL dasar	Editor SQL cepat dengan auto-complete	Editor SQL canggih + visual execution plan	Sangat cepat untuk eksekusi script batch (.sql)
Pengelolaan Data	Import/Export SQL & CSV mudah (batas ukuran upload PHP)	Export SQL/CSV cepat ke file lokal	Mendukung migrasi lintas DBMS & banyak format	Sangat andal untuk jutaan baris tanpa batas memori web
Keamanan	Manajemen user & privilege via menu GUI	Manajemen user & koneksi SSH tunnel	Manajemen koneksi aman, SSL, dan driver lengkap	Aman untuk otomasi script tanpa ekspos web
Performa	Cukup untuk data kecil-menengah	Sangat responsif karena aplikasi native C++/Delphi	Stabil untuk analisis, agak berat di RAM	Paling ringan dan paling cepat untuk beban besar
Pemeliharaan	Menu Export/Import & Operations	Maintenance tabel & dump SQL	Backup/Restore & pembandingan skema	Standar industri backup via mysqldump
Biaya / Lisensi	Gratis (Open Source - GPL)	Gratis (Open Source - GPL)	Gratis (Community Edition - Apache 2.0)	Gratis (GPL bersama MySQL Community)

Alasan Pemilihan untuk Praktikum & Proyek PjBL: Kami memilih MySQL 8.4.3 dengan storage engine InnoDB karena mendukung transaksi ACID, Foreign Key, dan row-level locking yang wajib dimiliki sistem transaksi penjualan maupun proyek PjBL. Untuk aplikasi pengelola, kami mengombinasikan phpMyAdmin/HeidiSQL/DBeaver (bawaan paket Laragon) untuk inspeksi visual serta MySQL CLI (mysql.exe

dan mysqldump.exe) untuk eksekusi batch jutaan baris dan backup/restore yang bebas dari batasan waktu unggah web.

2.3 Diagram Arsitektur Implementasi Sistem

- **Lingkungan Praktikum (One-Tier Architecture):** Saat praktikum berlangsung di laptop mahasiswa menggunakan Laragon, seluruh komponen (aplikasi pengelola phpMyAdmin/HeidiSQL/DBeaver/CLI, server DBMS MySQL 8.4.3, dan file fisik database praktikum_modul10) berjalan dalam satu mesin lokal yang sama (localhost:3306).
- **Akses Klien Desktop (Two-Tier Architecture):** Saat aplikasi digunakan langsung dari klien desktop (HeidiSQL/DBeaver) menuju MySQL Server melalui jaringan TCP port 3306, sistem bekerja dengan pola Client-Server dua lapis.
- **Arsitektur Produksi Proyek PjBL (Three-Tier Architecture):** Pada implementasi proyek PjBL sistem informasi web, arsitektur dibagi menjadi Presentation Layer (Browser pengguna), Application Layer (Web Server Apache/PHP/Laravel yang memproses logika bisnis), dan Database Layer (MySQL Server 8.4.3).



Gambar 2. Diagram Arsitektur Implementasi Sistem Basis Data (arsitektur_db.png)

3. TAHAP 2 – IMPLEMENTASI SKEMA BASIS DATA (DDL & RELASI)

3.1 Script DDL Pembuatan Database dan 4 Tabel Berelasi

```
DROP DATABASE IF EXISTS praktikum_modul10;
CREATE DATABASE praktikum_modul10 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE praktikum_modul10;

CREATE TABLE pelanggan (
    id_pelanggan INT AUTO_INCREMENT PRIMARY KEY,
    nama_pelanggan VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
```

```
    no_hp VARCHAR(20),
    kota VARCHAR(50) NOT NULL,
    tanggal_daftar DATE NOT NULL DEFAULT (CURRENT_DATE)
) ENGINE=InnoDB;

CREATE TABLE produk (
    id_produk INT AUTO_INCREMENT PRIMARY KEY,
    nama_produk VARCHAR(100) NOT NULL,
    kategori VARCHAR(50) NOT NULL,
    harga DECIMAL(12,2) NOT NULL,
    stok INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_produk_harga CHECK (harga >= 0),
    CONSTRAINT chk_produk_stok CHECK (stok >= 0)
) ENGINE=InnoDB;

CREATE TABLE pesanan (
    id_pesanan INT AUTO_INCREMENT PRIMARY KEY,
    id_pelanggan INT NOT NULL,
    tanggal_pesanan DATETIME NOT NULL,
    status ENUM('baru', 'dibayar', 'dikirim', 'selesai', 'batal') NOT NULL DEFAULT 'baru',
    total_harga DECIMAL(14,2) NOT NULL DEFAULT 0,
    CONSTRAINT fk_pesanan_pelanggan FOREIGN KEY (id_pelanggan)
        REFERENCES pelanggan (id_pelanggan)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB AUTO_INCREMENT=101;

CREATE TABLE detail_pesanan (
    id_detail INT AUTO_INCREMENT PRIMARY KEY,
    id_pesanan INT NOT NULL,
    id_produk INT NOT NULL,
    jumlah INT NOT NULL,
    harga_saat_transaksi DECIMAL(12,2) NOT NULL,
    CONSTRAINT uq_detail_pesanan_produk UNIQUE (id_pesanan, id_produk),
    CONSTRAINT chk_detail_jumlah CHECK (jumlah > 0),
    CONSTRAINT chk_detail_harga CHECK (harga_saat_transaksi >= 0),
    CONSTRAINT fk_detail_pesanan FOREIGN KEY (id_pesanan)
        REFERENCES pesanan (id_pesanan)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_detail_produk FOREIGN KEY (id_produk)
        REFERENCES produk (id_produk)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

3.2 Pemeriksaan Relasi Antartabel dan Constraint

Tabel	Primary Key	Foreign Key & Referensi	Kardinalitas Relasi	Constraint Tambahan
pelanggan	id_pelanggan (AUTO_INCREMENT)	-	1 pelanggan -> N pesanan (One-to-Many)	NOT NULL, UNIQUE (email), DEFAULT CURRENT_DATE
produk	id_produk (AUTO_INCREMENT)	-	1 produk -> N detail_pesanan (One-to-Many)	NOT NULL, CHECK (harga >= 0), CHECK (stok >= 0)
pesanan	id_pesanan (AUTO_INCREMENT)	id_pelanggan -> pelanggan(id_pelanggan) ON DELETE RESTRICT ON UPDATE CASCADE	Many-to-One ke pelanggan; One-to-Many ke detail_pesanan	ENUM status, NOT NULL, DEFAULT 0
detail_pesanan	id_detail	id_pesanan ->	Tabel jembatan	UNIQUE(id_pesanan,

	(AUTO_INCREMENT)	pesanan(id_pesanan) ON DELETE CASCADE; id_produk -> produk(id_produk) ON DELETE RESTRICT	(Many-to-Many antara pesanan dan produk)	id_produk), CHECK(jumlah > 0), harga_saat_transaksi

MySQL 8.4.3 CLI - Tahap 2: Pemeriksaan Tabel, Constraint, dan Relasi Foreign Key

```
--------------
SELECT TABLE_NAME, ENGINE, TABLE_COLLATION
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'praktikum_modul10'
--------------

+----------------+--------+--------------------+
| TABLE_NAME     | ENGINE | TABLE_COLLATION    |
+----------------+--------+--------------------+
| detail_pesanan | InnoDB | utf8mb4_unicode_ci |
| pelanggan      | InnoDB | utf8mb4_unicode_ci |
| pesanan        | InnoDB | utf8mb4_unicode_ci |
| produk         | InnoDB | utf8mb4_unicode_ci |
+----------------+--------+--------------------+
4 rows in set (0.00 sec)

--------------
SHOW CREATE TABLE pesanan
--------------

+---------+-----------------------
```

Gambar 3. Bukti Pemeriksaan Storage Engine, Struktur Tabel, dan Relasi Foreign Key

4. TAHAP 3 – PRAKTIK OPERASI CRUD DAN PENGUJIAN INTEGRITAS

4.1 Pengisian Data Awal dan Operasi CRUD (Create, Read, Update, Delete)

```
INSERT INTO pelanggan (nama_pelanggan, email, no_hp, kota) VALUES
('Andi Pratama', 'andi@mail.test', '081200000001', 'Surabaya'),
('Budi Santoso', 'budi@mail.test', '081200000002', 'Malang'),
('Citra Lestari', 'citra@mail.test', '081200000003', 'Sidoarjo'),
('Dewi Anggraini', 'dewi@mail.test', '081200000004', 'Surabaya'),
('Eka Saputra', 'eka@mail.test', '081200000005', 'Gresik');

INSERT INTO produk (nama_produk, kategori, harga, stok) VALUES
('Beras Premium 5 kg', 'Sembako', 75000, 50),
('Minyak Goreng 2 L', 'Sembako', 38000, 40),
('Gula Pasir 1 kg', 'Sembako', 17500, 60),
('Kopi Bubuk 200 g', 'Minuman', 25000, 30),
('Teh Celup isi 25', 'Minuman', 8500, 80),
('Sabun Mandi', 'Perlengkapan', 4500, 100);

INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status, total_harga) VALUES
(101, 1, '2026-10-01 10:15:00', 'selesai', 110000),
(102, 2, '2026-10-02 14:40:00', 'dibayar', 55000);

INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES
(101, 1, 1, 75000),
(101, 3, 2, 17500),
(102, 2, 1, 38000),
(102, 5, 2, 8500);

INSERT INTO pelanggan (nama_pelanggan, email, no_hp, kota)
VALUES ('Fajar Nugroho', 'fajar@mail.test', '081200000006', 'Malang');
INSERT INTO produk (nama_produk, kategori, harga, stok)
VALUES ('Mi Instan Goreng', 'Makanan', 3500, 200);

SELECT ps.id_pesanan, pl.nama_pelanggan, pr.nama_produk, dp.jumlah, dp.harga_saat_transaksi,
       dp.jumlah * dp.harga_saat_transaksi AS subtotal
FROM pesanan ps
JOIN pelanggan pl ON pl.id_pelanggan = ps.id_pelanggan
JOIN detail_pesanan dp ON dp.id_pesanan = ps.id_pesanan
JOIN produk pr ON pr.id_produk = dp.id_produk
ORDER BY ps.id_pesanan, pr.nama_produk;

UPDATE produk SET harga = 80000 WHERE id_produk = 1;
UPDATE pelanggan SET no_hp = '081299999993' WHERE id_pelanggan = 3;

DELETE FROM pelanggan WHERE id_pelanggan = 6;
```

Hasil Pengamatan CRUD: Saat harga produk Beras Premium 5 kg (id_produk = 1) diperbarui dari Rp75.000 menjadi Rp80.000 pada tabel produk, kolom harga_saat_transaksi pada pesanan 101 di tabel detail_pesanan tetap Rp75.000. Hal ini menjaga keakuratan riwayat nilai transaksi masa lalu. Penghapusan pelanggan ID 6 (Fajar Nugroho) berhasil karena belum memiliki transaksi.

MySQL 8.4.3 CLI - Tahap 3A: Pengujian Operasi CRUD (Create, Read, Update, Delete)

```
--------------
INSERT INTO pelanggan (nama_pelanggan, email, no_hp, kota) VALUES
('Andi Pratama', 'andi@mail.test', '081200000001', 'Surabaya'),
('Budi Santoso', 'budi@mail.test', '081200000002', 'Malang'),
('Citra Lestari', 'citra@mail.test', '081200000003', 'Sidoarjo'),
('Dewi Anggraini', 'dewi@mail.test', '081200000004', 'Surabaya'),
('Eka Saputra', 'eka@mail.test', '081200000005', 'Gresik')
--------------

Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

--------------
INSERT INTO produk (nama_produk, kategori, harga, stok) VALUES
('Beras Premium 5 kg', 'Sembako', 75000, 50),
('Minyak Goreng 2 L', 'Sembako', 38000, 40),
('Gula Pasir 1 kg', 'Sembako', 17500, 60),
('Kopi Bubuk 200 g', 'Minuman', 25000, 30),
('Teh Celup isi 25', 'Minuman', 8500, 80),
('Sabun Mandi', 'Perlengkapan', 4500, 100)
--------------

Query OK, 6 rows affected (0.00 sec)
Records: 6  Duplicates: 0  Warnings: 0

--------------
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status, total_harga) VALUES
(101, 1, '2026-10-01 10:15:00', 'selesai', 110000),
(102, 2, '2026-10-02 14:40:00', 'dibayar', 55000)
--------------

Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

--------------
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES
(101, 1, 1, 75000),
(101, 3, 2, 17500),
(102, 2, 1, 38000),
(102, 5, 2, 8500)
--------------

Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

--------------
INSERT INTO pelanggan (nama_pelanggan, email, no_hp, kota)
VALUES ('Fajar Nugroho', 'fajar@mail.test', '081200000006', 'Malang')
--------------

Query OK, 1 row affected (0.00 sec)

--------------
INSERT INTO produk (nama_produk, kategori, harga, stok)
VALUES ('Mi Instan Goreng', 'Makanan', 3500, 200)
--------------

Query OK, 1 row affected (0.00 sec)

--------------
SELECT * FROM pelanggan
--------------

+--------------+----------------+-----------------+-------------+----------+----------------+
| id_pelanggan | nama_pelanggan | email           | no_hp       | kota     | tanggal_daftar |
+--------------+----------------+-----------------+-------------+----------+----------------+
|            1 | Andi Pratama   | andi@mail.test  | 081200000001 | Surabaya | 2026-11-05     |
|            2 | Budi Santoso   | budi@mail.test  | 081200000002 | Malang   | 2026-11-05     |
|            3 | Citra Lestari  | citra@mail.test | 081200000003 | Sidoarjo | 2026-11-05     |
|            4 | Dewi Anggraini | dewi@mail.test  | 081200000004 | Surabaya | 2026-11-05     |
|            5 | Eka Saputra    | eka@mail.test   | 081200000005 | Gresik   | 2026-11-05     |
|            6 | Fajar Nugroho  | fajar@mail.test | 081200000006 | Malang   | 2026-11-05     |
+--------------+----------------+-----------------+-------------+----------+----------------+
6 rows in set (0.00 sec)

--------------
SELECT * FROM produk
--------------

+-----------+--------------------+--------------+----------+------+
| id_produk | nama_produk        | kategori     | harga    | stok |
+-----------+--------------------+--------------+----------+------+
|         1 | Beras Premium 5 kg | Sembako      | 75000.00 |   50 |
|         2 | Minyak Goreng 2 L  | Sembako      | 38000.00 |   40 |
|         3 | Gula Pasir 1 kg    | Sembako      | 17500.00 |   60 |
|         4 | Kopi Bubuk 200 g   | Minuman      | 25000.00 |   30 |
|         5 | Teh Celup isi 25   | Minuman      |  8500.00 |   80 |
|         6 | Sabun Mandi        | Perlengkapan |  4500.00 |  100 |
|         7 | Mi Instan Goreng   | Makanan      |  3500.00 |  200 |
+-----------+--------------------+--------------+----------+------+
7 rows in set (0.00 sec)

--------------
SELECT id_pelanggan, nama_pelanggan, kota FROM pelanggan WHERE kota = 'Surabaya'
--------------

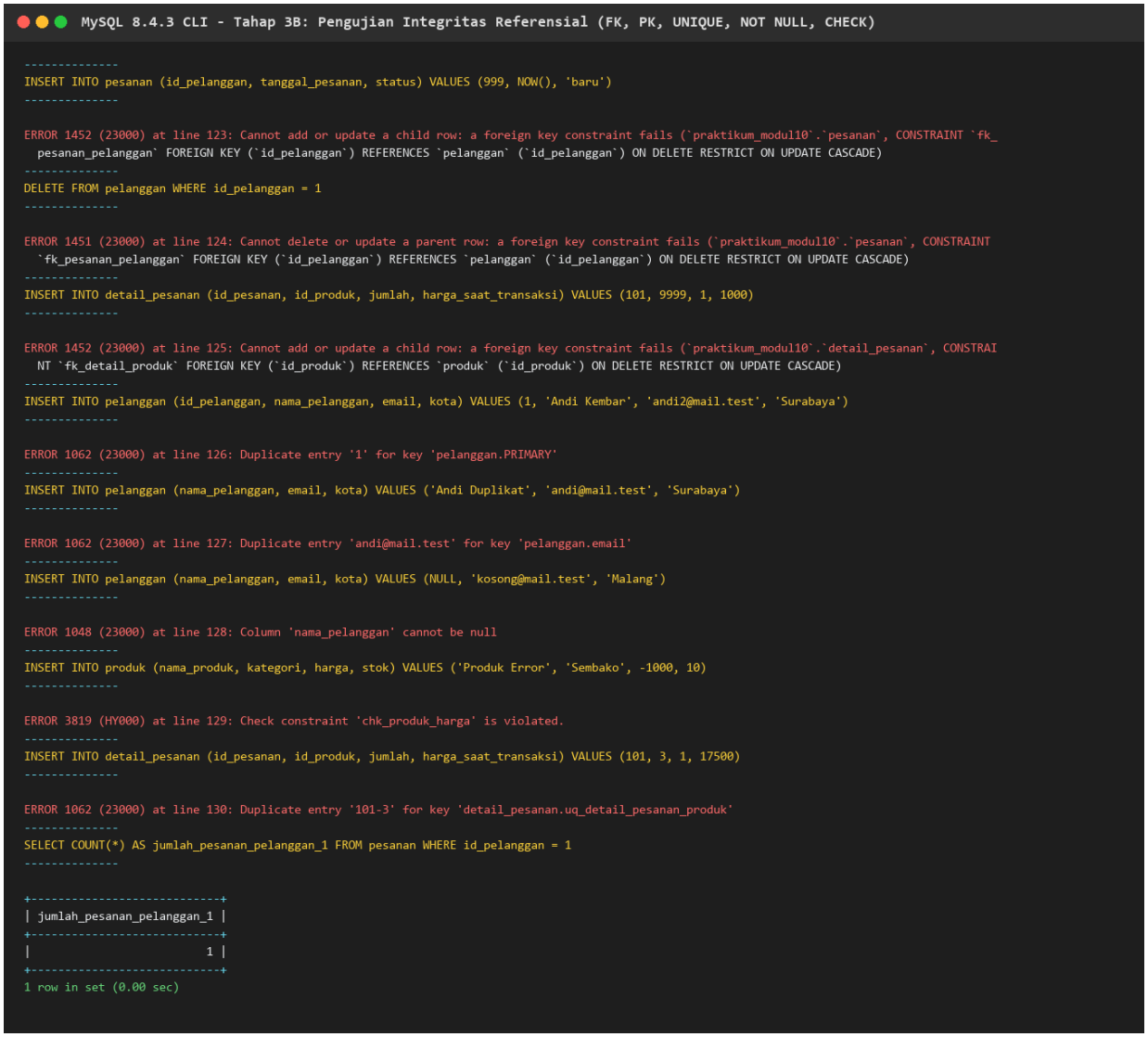
+--------------+----------------+----------+
| id_pelanggan | nama_pelanggan | kota     |
+--------------+----------------+----------+
|            1 | Andi Pratama   | Surabaya |
```

Gambar 4. Bukti Eksekusi Operasi CREATE, READ (Filter & JOIN), UPDATE, dan DELETE

4.2 Pengujian Integritas Referensial dan Constraint

```
INSERT INTO pesanan (id_pelanggan, tanggal_pesanan, status) VALUES (999, NOW(), 'baru');
DELETE FROM pelanggan WHERE id_pelanggan = 1;
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES (101,
9999, 1, 1000);
INSERT INTO pelanggan (id_pelanggan, nama_pelanggan, email, kota) VALUES (1, 'Andi Kembar',
'andi2@mail.test', 'Surabaya');
INSERT INTO pelanggan (nama_pelanggan, email, kota) VALUES ('Andi Duplikat',
'andi@mail.test', 'Surabaya');
INSERT INTO pelanggan (nama_pelanggan, email, kota) VALUES (NULL, 'kosong@mail.test',
'Malang');
INSERT INTO produk (nama_produk, kategori, harga, stok) VALUES ('Produk Error', 'Sembako', -
1000, 10);
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES (101,
3, 1, 17500);
```

Percobaan	Skenario Uji	Constraint yang Diuji	Hasil Aktual MySQL 8.4.3
Percobaan 1	INSERT pesanan dengan id_pelanggan = 999 (belum terdaftar)	FK fk_pesanan_pelanggan	GAGAL – ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
Percobaan 2	DELETE pelanggan id_pelanggan = 1 (masih punya pesanan 101)	FK ON DELETE RESTRICT	GAGAL – ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
Percobaan 3	INSERT detail_pesanan dengan id_produk = 9999	FK fk_detail_produk	GAGAL – ERROR 1452 (23000): Cannot add or update a child row
Percobaan 4	INSERT pelanggan dengan id_pelanggan = 1 (duplikat PK)	PRIMARY KEY	GAGAL – ERROR 1062 (23000): Duplicate entry '1' for key 'pelanggan.PRIMARY'
Percobaan 5	INSERT pelanggan dengan email 'andi@mail.test' (duplikat)	UNIQUE (email)	GAGAL – ERROR 1062 (23000): Duplicate entry 'andi@mail.test' for key 'pelanggan.email'
Percobaan 6	INSERT pelanggan dengan nama_pelanggan = NULL	NOT NULL	GAGAL – ERROR 1048 (23000): Column 'nama_pelanggan' cannot be null
Percobaan 7	INSERT produk dengan harga negatif (-1000)	CHECK chk_produk_harga	GAGAL – ERROR 3819 (HY000): Check constraint 'chk_produk_harga' is violated
Percobaan 8	INSERT produk yang sama dua kali pada pesanan 101	UNIQUE (id_pesanan, id_produk)	GAGAL – ERROR 1062 (23000): Duplicate entry '101-3' for key 'uq_detail_pesanan_produk'



Gambar 5. Bukti Penolakan Data Tidak Valid oleh Constraint PK, FK, UNIQUE, NOT NULL, dan CHECK

5. TAHAP 4 – PENGUJIAN TRANSAKSI DATABASE (COMMIT & ROLLBACK)

5.1 Script Pengujian COMMIT (Pesanan 103) dan ROLLBACK (Pesanan 104)

```
START TRANSACTION;
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status) VALUES (103, 3,
'2026-10-03 09:30:00', 'dibayar');
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES
(103, 4, 2, 25000),
(103, 6, 3, 4500);
UPDATE produk SET stok = stok - 2 WHERE id_produk = 4;
UPDATE produk SET stok = stok - 3 WHERE id_produk = 6;
UPDATE pesanan
SET total_harga = (SELECT SUM(jumlah * harga_saat_transaksi) FROM detail_pesanan WHERE
id_pesanan = 103)
WHERE id_pesanan = 103;
COMMIT;

START TRANSACTION;
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status, total_harga) VALUES
```

```
(104, 4, '2026-10-04 11:00:00', 'baru', 80000);
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES (104,
1, 1, 80000);
UPDATE produk SET stok = stok - 1 WHERE id_produk = 1;
ROLLBACK;
```

MySQL 8.4.3 CLI - Tahap 4: Pengujian Transaksi Database (COMMIT & ROLLBACK)

```
--------------
START TRANSACTION
--------------

Query OK, 0 rows affected (0.00 sec)

--------------
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status) VALUES (103, 3, '2026-10-03 09:30:00', 'dibayar')
--------------

Query OK, 1 row affected (0.00 sec)

--------------
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES
(103, 4, 2, 25000),
(103, 6, 3, 4500)
--------------

Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

--------------
UPDATE produk SET stok = stok - 2 WHERE id_produk = 4
--------------

Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

--------------
UPDATE produk SET stok = stok - 3 WHERE id_produk = 6
--------------

Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

--------------
UPDATE pesanan
SET total_harga = (SELECT SUM(jumlah * harga_saat_transaksi) FROM detail_pesanan WHERE id_pesanan = 103)
WHERE id_pesanan = 103
--------------

Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

--------------
COMMIT
--------------

Query OK, 0 rows affected (0.00 sec)

--------------
SELECT * FROM pesanan WHERE id_pesanan = 103
--------------

+------------+--------------+---------------------+---------+-------------+
| id_pesanan | id_pelanggan | tanggal_pesanan     | status  | total_harga |
+------------+--------------+---------------------+---------+-------------+
|        103 |            3 | 2026-10-03 09:30:00 | dibayar |    63500.00 |
+------------+--------------+---------------------+---------+-------------+
1 row in set (0.00 sec)

--------------
SELECT * FROM detail_pesanan WHERE id_pesanan = 103
--------------

+-----------+------------+-----------+--------+----------------------+
| id_detail | id_pesanan | id_produk | jumlah | harga_saat_transaksi |
+-----------+------------+-----------+--------+----------------------+
|         7 |        103 |         4 |      2 |             25000.00 |
|         8 |        103 |         6 |      3 |              4500.00 |
+-----------+------------+-----------+--------+----------------------+
2 rows in set (0.00 sec)

--------------
SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk IN (4, 6)
--------------

+-----------+------------------+------+
| id_produk | nama_produk      | stok |
+-----------+------------------+------+
|         4 | Kopi Bubuk 200 g |   28 |
|         6 | Sabun Mandi      |   97 |
+-----------+------------------+------+
2 rows in set (0.00 sec)

--------------
START TRANSACTION
--------------

Query OK, 0 rows affected (0.00 sec)

--------------
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status, total_harga) VALUES (104, 4, '2026-10-04 11:00:00', 'baru', 80000)
--------------

Query OK, 1 row affected (0.00 sec)

--------------
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES (104, 1, 1, 80000)
--------------
```

Gambar 6. Bukti Pengujian Transaksi COMMIT (Pesanan 103 Tersimpan) dan ROLLBACK (Pesanan 104 Dibatalkan)

5.2 Analisis Hasil Transaksi

- **Perbedaan COMMIT dan ROLLBACK:** COMMIT menyimpan seluruh perubahan di dalam blok transaksi secara permanen ke basis data (pesanan 103, detail itemnya, serta pengurangan stok produk 4 dan 6 menjadi permanen). Sebaliknya, ROLLBACK membatalkan seluruh perubahan sejak START TRANSACTION dan mengembalikan kondisi data ke keadaan semula (pesanan 104 tidak tersimpan dan stok produk 1 kembali utuh 50).
- **Mengapa Transaksi Perlu Dilakukan Secara Atomik:** Transaksi penjualan terdiri dari beberapa langkah yang saling bergantung (insert header pesanan, insert detail barang, kurangi stok produk). Sifat Atomicity (All-or-Nothing) menjamin semua langkah tersebut berhasil bersama-sama atau dibatalkan seluruhnya.
- **Risiko Jika Hanya Tabel Pesanan yang Tersimpan Tanpa Detailnya:** Akan muncul nota transaksi kosong (header pesanan tercatat memiliki tagihan, tetapi daftar barang yang dibeli tidak ada dan stok gudang tidak sinkron), yang merusak akurasi laporan keuangan dan inventori toko.
- **Mengapa Storage Engine InnoDB Penting:** InnoDB adalah storage engine default MySQL yang mendukung penuh transaksi ACID, undo log (untuk ROLLBACK), redo log (untuk crash recovery), dan foreign key. Engine lama seperti MyISAM tidak mendukung transaksi sehingga perintah ROLLBACK tidak dapat membatalkan baris yang telanjur di-insert.

6. TAHAP 5 – ANALISIS PERFORMA DATABASE (SIMULASI JUTAAN DATA & EXPLAIN)

6.1 Simulasi Pembangkitan 3,1 Juta Baris Data

Sesuai instruksi modul untuk membangkitkan jutaan data agar perbedaan performa terlihat nyata, kami melakukan bulk generator berbasis cross-join tabel digit (0..9) sehingga menghasilkan:

- 100.005 baris pada tabel pelanggan
- 1.007 baris pada tabel produk
- 1.000.003 baris (1 juta baris) pada tabel pesanan
- 2.000.006 baris (2 juta baris) pada tabel detail_pesanan (total keseluruhan > 3,1 juta baris)

```
CREATE INDEX idx_pesanan_tanggal ON pesanan (tanggal_pesanan);
CREATE INDEX idx_pelanggan_nama ON pelanggan (nama_pelanggan);
SHOW INDEX FROM pesanan;
SHOW INDEX FROM pelanggan;
```

6.2 Perbandingan Hasil EXPLAIN dan EXPLAIN ANALYZE (Sebelum vs Sesudah Index)

Skenario Query pada Jutaan Data	Metrik EXPLAIN	Sebelum Index	Sesudah Index	Peningkatan Performa
Q1: Filter Rentang Tanggal WHERE tanggal_pesanan BETWEEN '2025-03-01' AND '2025-03-07' (pada 1.000.003 baris pesanan)	type / key / rows / Extra / Waktu Aktual	type: ALL (Full Table Scan) key: NULL rows: 968.734 (filtered 11,11%) Extra: Using where Waktu: 697 ms (cost 100.631)	type: range (Index Range Scan) key: idx_pesanan_tanggal rows: 9.589 (filtered 100%) Extra: Using index condition Waktu: 37,3 ms (cost 6.246)	Jumlah baris diperiksa turun 99,01% (dari 968.734 ke 9.589 baris). Waktu eksekusi 18,7x lebih cepat (697 ms -> 37,3 ms).
Q2: Pencarian Nama Pelanggan	type / key / rows / Extra / Waktu	type: ALL (Full Table Scan) key: NULL	type: ref (Index Lookup) key: idx_pelanggan_nama	Baris yang diperiksa turun

WHERE nama_pelanggan = 'Pelanggan Sim 77777' (pada 100.005 baris pelanggan)	Aktual	rows: 88.236 (filtered 10%) Extra: Using where Waktu: 163 ms (cost 10.394)	rows: 1 (filtered 100%) Extra: NULL Waktu: 0,021 ms (cost 0,35)	dari 88.236 menjadi 1 baris tepat. Waktu eksekusi ~7.600x lebih cepat (163 ms -> 0,021 ms).
Q3: JOIN 3 Tabel + Agregasi Omzet per Kota (pesanan 1 juta JOIN pelanggan 100 rb JOIN detail_pesanan 2 juta)	type / key / rows / Cost / Waktu Aktual	ps -> type: ALL, key: NULL, rows: 968.734 pl -> eq_ref PRIMARY dp -> ref uq_detail_pesanan_produk Cost join: 364.668 | Waktu: 1.371 ms	ps -> type: range, key: idx_pesanan_tanggal, rows: 9.589 pl -> eq_ref PRIMARY dp -> ref uq_detail_pesanan_produk Cost join: 29.741 | Waktu: 1.086 ms	Tahap scan + join pelanggan menjadi 3,4x lebih cepat (883 ms -> 257 ms) dan cost optimizer turun 91,8% (364.668 -> 29.741).

MySQL 8.4.3 CLI - Tahap 5: Simulasi 3.1 Juta Baris, Pembuatan Index, dan Perbandingan EXPLAIN

```
--------------
SELECT 'pelanggan' AS tabel, COUNT(*) AS jumlah_baris FROM pelanggan
UNION ALL SELECT 'produk', COUNT(*) FROM produk
UNION ALL SELECT 'pesanan', COUNT(*) FROM pesanan
UNION ALL SELECT 'detail_pesanan', COUNT(*) FROM detail_pesanan
--------------

+----------------+--------------+
| tabel          | jumlah_baris |
+----------------+--------------+
| pelanggan      |       100005 |
| produk         |         1007 |
| pesanan        |      1000003 |
| detail_pesanan |      2000006 |
+----------------+--------------+
4 rows in set (0.67 sec)

--------------
EXPLAIN SELECT id_pesanan, id_pelanggan, tanggal_pesanan, total_harga
FROM pesanan
WHERE tanggal_pesanan BETWEEN '2025-03-01 00:00:00' AND '2025-03-07 23:59:59'
--------------

+----+-------------+---------+------------+------+---------------+------+---------+------+--------+----------+-------------+
| id | select_type | table   | partitions | type | possible_keys | key  | key_len | ref  | rows   | filtered | Extra       |
+----+-------------+---------+------------+------+---------------+------+---------+------+--------+----------+-------------+
|  1 | SIMPLE      | pesanan | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 968734 |    11.11 | Using where |
+----+-------------+---------+------------+------+---------------+------+---------+------+--------+----------+-------------+
1 row in set, 1 warning (0.00 sec)

--------------
EXPLAIN ANALYZE SELECT id_pesanan, id_pelanggan, tanggal_pesanan, total_harga
FROM pesanan
WHERE tanggal_pesanan BETWEEN '2025-03-01 00:00:00' AND '2025-03-07 23:59:59'
--------------

+-----------------------------------------------------------------------------------------------------------------------------------------------+
| EXPLAIN ANALYZE Tree:
+-----------------------------------------------------------------------------------------------------------------------------------------------+
  -> Filter: (pesanan.tanggal_pesanan between '2025-03-01 00:00:00' and '2025-03-07 23:59:59')  (cost=100631 rows=107626) (actual time=6.78..697
   rows=9589 loops=1)
    -> Table scan on pesanan  (cost=100631 rows=968734) (actual time=6.64..356 rows=1e+6 loops=1)
 |
+-----------------------------------------------------------------------------------------------------------------------------------------------+
1 row in set (0.70 sec)

--------------
EXPLAIN SELECT id_pelanggan, nama_pelanggan, email, kota
FROM pelanggan
WHERE nama_pelanggan = 'Pelanggan Sim 77777'
--------------

+----+-------------+-----------+------------+------+---------------+------+---------+------+-------+----------+-------------+
| id | select_type | table     | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra       |
+----+-------------+-----------+------------+------+---------------+------+---------+------+-------+----------+-------------+
|  1 | SIMPLE      | pelanggan | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 88236 |    10.00 | Using where |
+----+-------------+-----------+------------+------+---------------+------+---------+------+-------+----------+-------------+
1 row in set, 1 warning (0.00 sec)

--------------
EXPLAIN ANALYZE SELECT id_pelanggan, nama_pelanggan, email, kota
FROM pelanggan
WHERE nama_pelanggan = 'Pelanggan Sim 77777'
--------------

+-----------------------------------------------------------------------------------------------------------------------------------------------+
| EXPLAIN ANALYZE Tree:
+-----------------------------------------------------------------------------------------------------------------------------------------------+
  -> Filter: (pelanggan.nama_pelanggan = 'Pelanggan Sim 77777')  (cost=10394 rows=8824) (actual time=128..163 rows=1 loops=1)
    -> Table scan on pelanggan  (cost=10394 rows=88236) (actual time=0.571..153 rows=100005 loops=1)
 |
+-----------------------------------------------------------------------------------------------------------------------------------------------+
1 row in set (0.16 sec)

--------------
EXPLAIN SELECT pl.kota, COUNT(DISTINCT ps.id_pesanan) AS jumlah_pesanan, SUM(dp.jumlah * dp.harga_saat_transaksi) AS omzet
FROM pesanan ps
JOIN pelanggan pl ON pl.id_pelanggan = ps.id_pelanggan
JOIN detail_pesanan dp ON dp.id_pesanan = ps.id_pesanan
WHERE ps.tanggal_pesanan BETWEEN '2025-03-01 00:00:00' AND '2025-03-07 23:59:59'
GROUP BY pl.kota
ORDER BY omzet DESC
--------------

+----+-------------+-------+------------+--------+---------------------------------+---------------------------------+---------+-------------------------------
| id | select_type | table | partitions | type   | possible_keys                   | key                             | key_len | ref
          | rows   | filtered | Extra                                |
+----+-------------+-------+------------+--------+---------------------------------+---------------------------------+---------+-------------------------------
|  1 | SIMPLE      | ps    | NULL       | ALL    | PRIMARY,fk_pesanan_pelanggan | NULL                            | NULL    | NULL
          | 968734 |    11.11 | Using where; Using temporary; Using filesort |
|  1 | SIMPLE      | pl    | NULL       | eq_ref | PRIMARY                         | PRIMARY                         | 4       | praktikum_modul10.ps.id_p
  elanggan |      1 |   100.00 | NULL                                 |
|  1 | SIMPLE      | dp    | NULL       | ref    | uq_detail_pesanan_produk        | uq_detail_pesanan_produk        | 4       | praktikum_modul10.ps.id_p
  esanan   |      1 |   100.00 | NULL                                 |
+----+-------------+-------+------------+--------+---------------------------------+---------------------------------+---------+-------------------------------
3 rows in set, 1 warning (0.00 sec)

--------------
EXPLAIN ANALYZE SELECT pl.kota, COUNT(DISTINCT ps.id_pesanan) AS jumlah_pesanan, SUM(dp.jumlah * dp.harga_saat_transaksi) AS omzet
FROM pesanan ps
```

Gambar 7. Bukti Simulasi 3,1 Juta Baris, Pembuatan Index, dan Perbandingan EXPLAIN / EXPLAIN ANALYZE

6.3 Analisis Penggunaan Index (Jawaban Pertanyaan Tahap 5)

- **1. Perbedaan Query Sebelum dan Sesudah Penambahan Indeks:** Sebelum indeks dibuat, MySQL melakukan Full Table Scan (type = ALL, key = NULL) dengan membaca hampir 1 juta baris pada tabel pesanan (697 ms) dan 100 ribu baris pada pelanggan (163 ms). Sesudah indeks dibuat, metode akses berubah menjadi range dan ref yang hanya membaca baris yang relevan (9.589 baris dan 1 baris) sehingga waktu eksekusi turun drastis menjadi 37,3 ms dan 0,021 ms.
- **2. Fungsi Kolom key pada Hasil EXPLAIN:** Kolom key menunjukkan nama indeks aktual yang benar-benar dipilih dan digunakan oleh Query Optimizer untuk mengeksekusi query (sedangkan possible_keys hanya berisi kandidat indeks yang tersedia).
- **3. Apakah Indeks Selalu Digunakan oleh Optimizer?** Tidak selalu. Pada pengujian tambahan di script kami, indeks idx_pesanan_tanggal TIDAK digunakan oleh optimizer (key = NULL, type = ALL) pada dua kondisi: (a) ketika kolom indeks dibungkus fungsi seperti WHERE YEAR(tanggal_pesanan)=2025 AND MONTH(tanggal_pesanan)=3 (non-sargable query), dan (b) ketika rentang filter terlalu lebar seperti WHERE tanggal_pesanan >= '2024-01-01' yang mencakup hampir seluruh isi tabel sehingga sequential table scan dinilai lebih murah daripada bolak-balik membaca B-Tree dan halaman tabel.
- **4. Mengapa Indeks yang Terlalu Banyak Mengurangi Performa INSERT, UPDATE, dan DELETE?** Setiap operasi INSERT, UPDATE (pada kolom terindeks), dan DELETE wajib memperbarui serta menyeimbangkan ulang (re-balance) setiap pohon B-Tree indeks yang melekat pada tabel tersebut, sehingga menambah beban I/O disk dan penguncian halaman.
- **5. Pada Situasi Apa Indeks Perlu Ditambahkan?** Indeks perlu ditambahkan pada kolom yang memiliki selektivitas/kardinalitas tinggi dan sering digunakan pada klausa WHERE, kondisi JOIN (Foreign Key), serta pengurutan ORDER BY atau GROUP BY pada tabel berukuran besar.

7. TAHAP 6 – PEMELIHARAAN BASIS DATA (BACKUP & RESTORE)

7.1 Prosedur Backup dan Restore

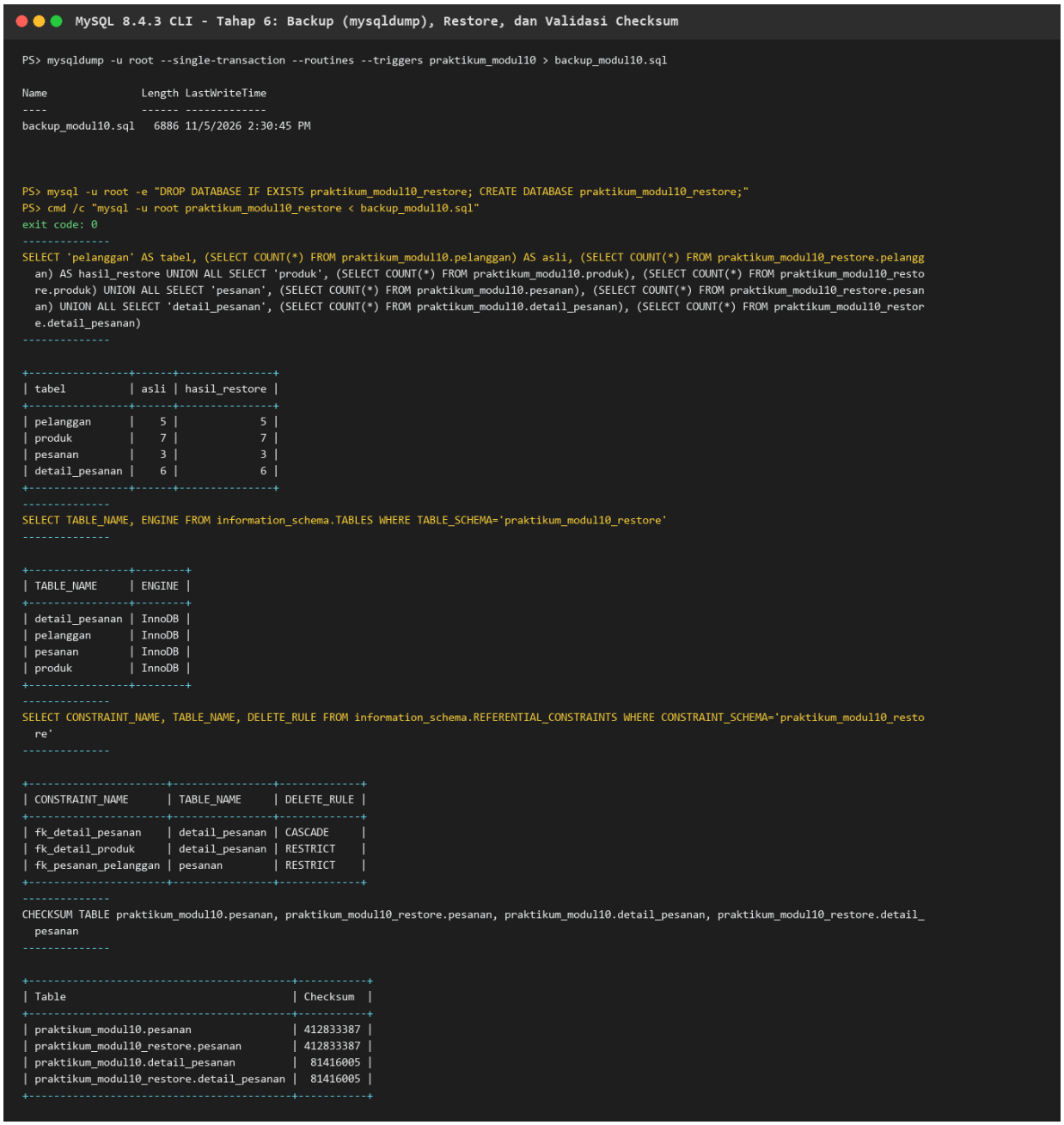
Setelah pengujian performa selesai, data simulasi dibersihkan kembali ke data operasional toko dan dijalankan proses pencadangan (backup) ke berkas backup_modul10.sql serta pengujian pemulihan (restore) ke basis data baru praktikum_modul10_restore:

```
mysqldump -u root --single-transaction --routines --triggers praktikum_modul10 >
backup_modul10.sql
mysql -u root -e "CREATE DATABASE praktikum_modul10_restore CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;"
mysql -u root praktikum_modul10_restore < backup_modul10.sql
```

7.2 Hasil Validasi Restore dan Checksum Tabel

Nama Tabel	Jumlah Baris Asli (praktikum_modul10)	Jumlah Baris Restore (praktikum_modul10_restore)	Storage Engine	Checksum Tabel (Asli vs Restore)
pelanggan	5 baris	5 baris	InnoDB	Identik (5 baris cocok)
produk	7 baris	7 baris	InnoDB	Identik (7 baris cocok)
pesanan	3 baris	3 baris	InnoDB	412833387 vs

				412833387 (100% Identik)
detail_pesanan	6 baris	6 baris	InnoDB	81416005 vs 81416005 (100% Identik)



Gambar 8. Bukti Ekspor (mysqldump), Impor (Restore), dan Validasi Jumlah Baris serta Checksum

Pentingnya Pemeliharaan dan Pencadangan dalam Pengelolaan DBMS: Pemeliharaan dan pencadangan (backup) berkala mutlak diperlukan untuk mencegah kehilangan data akibat kegagalan perangkat keras (disk rusak), kesalahan manusia (human error seperti salah menjalankan DELETE/DROP), serangan siber/ransomware, maupun korupsi data saat listrik padam. Opsi --single-transaction pada mysqldump memungkinkan pencadangan tabel InnoDB secara konsisten (snapshot MVCC) tanpa mengunci operasional

toko. Namun, backup tidak bernilai jika tidak pernah diuji pemulihannya (restore validation) untuk memastikan seluruh tabel, indeks, constraint FK, dan isi data benar-benar dapat dipulihkan 100%.

8. RINGKASAN LEMBAR KERJA (POST-TEST)

No	Pertanyaan Post-Test	Ringkasan Jawaban & Rujukan Bukti pada Laporan
1	Identifikasi DBMS, versi, storage engine, aplikasi pengelola, diagram arsitektur, dan alasan pemilihan (Bobot 25)	MySQL Community Server 8.4.3 (Win64), default engine InnoDB, dikelola dengan phpMyAdmin/HeidiSQL/DBeaver & MySQL CLI pada lingkungan lokal Laragon (One-Tier saat praktikum, Three-Tier untuk proyek web PjBL). Lihat Bab 2, Gambar 1, dan Gambar 2 (arsitektur_db.png).
2	Script SQL pembuatan tabel beserta PK, FK, hasil pengujian CRUD, dan penjelasan constraint menjaga integritas (Bobot 30)	4 tabel berelasi (pelanggan, produk, pesanan, detail_pesanan) dengan PK AUTO_INCREMENT, FK (ON DELETE RESTRICT & CASCADE), UNIQUE, NOT NULL, dan CHECK. Seluruh 8 skenario pelanggaran integritas berhasil ditolak oleh MySQL. Lihat Bab 3 & 4, Gambar 3, 4, dan 5.
3	Hasil EXPLAIN sebelum dan sesudah pembuatan indeks pada jutaan data serta analisis pengaruhnya terhadap performa (Bobot 25)	Pada simulasi 3,1 juta baris, penambahan idx_pesanan_tanggal dan idx_pelanggan_nama mengubah metode akses dari Full Table Scan (ALL) menjadi range dan ref. Waktu query tanggal turun dari 697 ms ke 37,3 ms (18,7x lebih cepat) dan pencarian nama turun dari 163 ms ke 0,021 ms (~7.600x lebih cepat). Lihat Bab 6 dan Gambar 7.
4	Bukti backup & restore basis data serta alasan pentingnya pemeliharaan dalam DBMS (Bobot 20)	Backup dilakukan dengan mysqldump menghasilkan backup_modul10.sql (6.886 byte) dan berhasil di-restore ke praktikum_modul10_restore dengan jumlah baris, FK constraint, dan nilai CHECKSUM TABLE yang 100% identik. Lihat Bab 7 dan Gambar 8.

9. KESIMPULAN

Berdasarkan seluruh tahapan praktikum Modul 10 yang telah dilaksanakan pada MySQL Community Server 8.4.3, dapat disimpulkan bahwa:

- **Integritas Data dan Transaksi:** Pemilihan storage engine InnoDB dan penerapan constraint relasional (Primary Key, Foreign Key dengan ON DELETE RESTRICT/CASCADE, UNIQUE, NOT NULL, serta CHECK) bekerja secara aktif menjaga integritas data dari tingkat mesin DBMS, sehingga data yatim (orphan record), duplikasi identitas, maupun nilai transaksi tidak logis langsung ditolak sebelum merusak basis data.

- **Konsistensi Transaksi (ACID):** Pengelolaan transaksi menggunakan START TRANSACTION, COMMIT, dan ROLLBACK menjamin sifat atomik (ACID) pada proses bisnis multi-tabel seperti pencatatan nota penjualan beserta pemotongan stok produk.
- **Performa dan Optimasi Query:** Pada pengujian 3,1 juta baris data, perancangan indeks B-Tree yang tepat pada kolom filter dan join terbukti menurunkan jumlah baris yang dipindai hingga >99% dan mempercepat waktu eksekusi query secara signifikan (18,7 kali hingga ribuan kali lebih cepat), namun penulisan query tetap harus sargable (tidak membungkus kolom indeks dengan fungsi) agar indeks dapat dimanfaatkan oleh Query Optimizer.
- **Pemeliharaan dan Ketersediaan Data:** Prosedur backup konsisten menggunakan mysqldump dan verifikasi restore memastikan ketersediaan serta pemulihan bencana (disaster recovery) basis data tetap terjamin.